RL Cheatsheet

"Calcola copertura minima" $\implies$ *Mappe K o Quine-MCK*

Es.1 Espressioni, mappe di Karnaugh e Teorema del consenso

- Semplificazione di funzioni booleane tramite mappe di Karnaugh
  1. Regole per implicanti:

     Valido? $\rightarrow 2^n$ celle adiacenti (no num.dispari, no diagonali)

     Implicante primo? NON riesco a estendere senza toccare 0? $\rightarrow$ è **implicante primo**

     Implicante primo essenziale? Se copre un 1 che non può essere coperto da altri implicanti primi, allora è essenziale
  2. Ricavo i mintermini se non sono dati tramite tabella di verità (sono quelli con 1 come valore di uscita)
  3. Dato l'ON_{set} di f riporto nella mappa di Karnaugh tutti gli 1 o X nelle corrispondenti cellette della tabella
  4. Raggruppo per multipli di 2^n i gruppi di 1 e X adiacenti, prediligendo quelli più grandi possibile

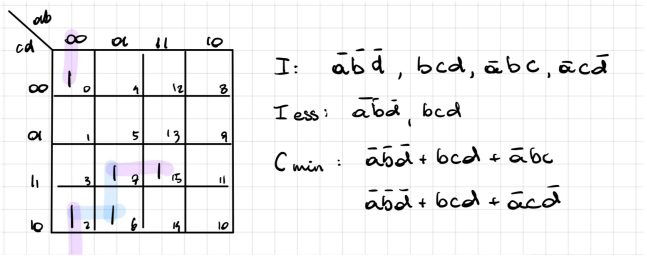
     La mappa è da pensarsi come la carina geografica: i "bordi" adiacenti fra di loro, se ho quindi 1 a tutti e quattro gli angoli posso raggruppare quei 4 e contarli come unico gruppo.
  5. Scrivo i valori dei gruppi seguendo il metodo:
     (a) Guardo la riga dove è presente il gruppo
     (b) guardo i valori delle celle sopra: se $AB: 00 \rightarrow 01$ **prendo il letterale del bit che NON cambia** $(A: 0 \rightarrow 0, B: 0 \rightarrow 1)$ e lo riporto seguendo la regola:
        i. A se $1 \rightarrow 1$
        ii. $\overline{A}$ se $0 \rightarrow 0$
     (c) una volta identificati tutti i raggruppamenti, gli **implicanti primi ESSENZIALI** sono quelli che sono gli unici a coprire uno specifico 1, che non può essere coperto dagli altri raggruppamenti; tutti gli altri sono implicanti primi
  6. Scrivo la **copertura minima** come **SoP (somma di prodotti)** costituita da TUTTI gli IMPLICANTI ESSENZIALI + gli implicanti primi che coprono gli 1 mancanti all'interno della mappa.

Esercizio 2.4 Si consideri la seguente funzione f definita attraverso il suo ON_{set}:

$$ON_{set} = \{m0, m2, m6, m7, m15\}$$

Facendo uso della sua mappa di Karnaugh, calcolare:

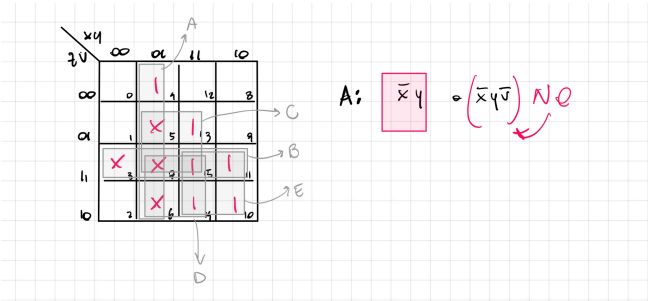
1. Implicanti primi
2. Implicanti primi essenziali
3. Copertura minima
4. Dire se la copertura minima trovata è unica
5. Se la copertura minima trovata non è unica, calcolare un'altra copertura minima



In viola: IMPLICANTI PRIMI ESSENZIALI

In azzurro: IMPLICANTI PRIMI

La mappa è da pensarsi come la carina geografica: i "bordi" adiacenti fra di loro, se ho quindi 1 a tutti e quattro gli angoli posso raggruppare quei 4 e contarli come unico gruppo.



Ordine mintermini in Mappe K

0	4	12	8
1	5	13	9
3	7	15	11
2	6	14	10

Regola Generale: Raggruppamento e Numero di Letterali

*In una mappa di Karnaugh con **M variabili**, un gruppo di 2^n celle contigue **elimina n variabili** dall'espressione, lasciando M - n **letterali** nell'implicante finale.*

1. **Mappa a 3 variabili (M = 3, quindi 8 celle totali):**
   - **1 cella** (raggruppamento 2^0): non elimino nessuna variabile, l'implicante ha 3 letterali.
   - 2 celle (raggruppamento 2^1): elimino 1 variabile, l'implicante ha $3 - 1 = 2$ letterali.
   - **4 celle** (raggruppamento 2^2): elimino 2 variabili, l'implicante ha $3 - 2 = 1$ letterale.
   - **8 celle** (raggruppamento 2^3): elimino 3 variabili, l'implicante ha $3 - 3 = 0$ letterali (funzione sempre 1).
2. **Mappa a 4 variabili (M = 4, quindi 16 celle totali):**
   - **1 cella:** non elimino nessuna variabile, 4 letterali.
   - **2 celle:** elimino 1 variabile, 3 letterali.
   - **4 celle:** elimino 2 variabili, 2 letterali.
   - **8 celle:** elimino 3 variabili, 1 letterale.
   - **16 celle:** elimino 4 variabili, 0 letterali (funzione sempre 1).

- Semplificazione di funzioni booleane tramite proprietà dell'algebra

Algebra di Commutazione: proprietà

Errore tipico:

$$\overline{(AB)} = \overline{A \cdot B} = \overline{A} + \overline{B}$$

1. elemento neutro
   - a+1=1 $a * 0 = 0$ (elemento nullo o assorbente)
   - a+0=a $a * 1 = a$ (identità)
2. idempotenza
   - a+a=a $a * a = a$
3. inverso o complemento
   - a+a'=1 $a^* a' = 0$
4. commutativa
   - a+b=b+a $a * b = b * a$
5. associativa
   - $a + (b + c) = (a + b) + c$
     $a * (b * c) = (a * b) * c$
6. distributiva
   - $a * (b + c) = a * b + a * c$
     $a + (b * c) = (a + b) * (a + c)$
   - Si osservi la priorità delle operazioni
7. assorbimento
   - a+(a*b)=a $a * (a + b) = a$

7-b **assorbimento del complemento**

- $a + \bar{a}b = a + b$ $a * (a' + b) = a * b$

8. **OCCHIO!** *questo non è un assorbimento!*

$$B\bar{C} + \bar{B}D \neq B\bar{C} + D$$

È infatti diverso dalla forma

$$x + \bar{x}y = x + y$$

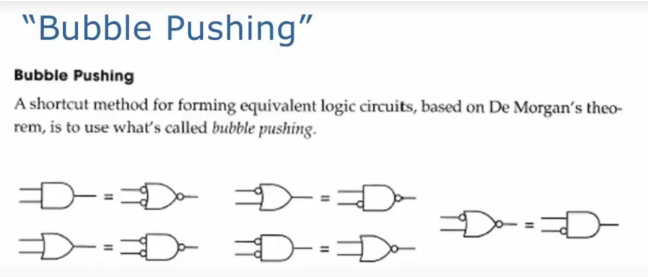
9. Questo é **corretto**:

$$A + \bar{A}BC = A + BC$$

10. Leggi di De Morgan
    - (a+b)'=a'*b' $(a * b)' = a' + b'$
11. Consenso
    - $a * b + a' * c = a"b + a' * c$
      $ab + a'c + bc = ab + a'c$
    - $(a + b) * (a' + c) * (b + c) = (a + b) * (a' + c)$
      $(a + b)(a' + c)(b + c) = (a + b)(a' + c)$

Bubble-Push

Si usa per **trovare direttamente** a partire dalla rete, il **suo complemento**

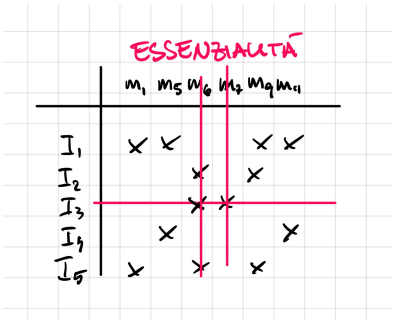


Es.2 Quine-McKlusky

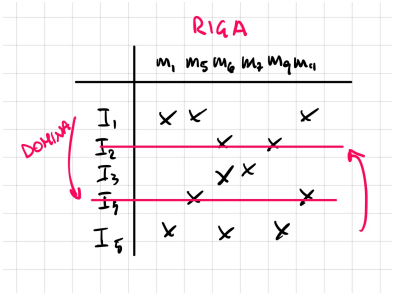
1. Scrivo in binario ON_{Set} e DC_{Set}
2. Raggruppo in base al numero di "1" presenti
3. Confronto bit a bit con quelli del gruppo successivo e raggruppo se la distanza di hamming è $H_d = 1$

Singola funzione di Output

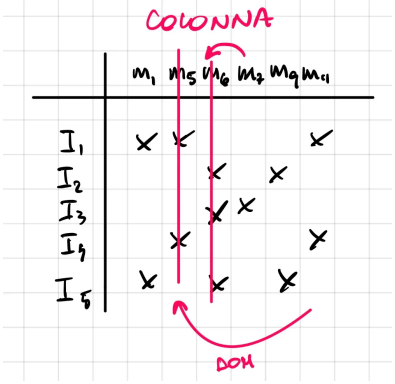
1. Segno i mintermini che ho usato per il raggruppamento, e itero il procedimento fino a quando non posso più raggruppare
2. Segno con lettere maiuscole i mintermini che non possono essere più raggruppati. Questi rappresentano i miei **IMPLICANTI PRIMI**
3. Riporto in una tabella gli implicanti primi come segue
4. Eseguo semplificazioni come segue:
   (a) **Essenzialità**:
   1. Aggiungo implicante a copertura (ess. primaria/secondaria,etc...)
   2. Elimino dalla tabella tutte le colonne che hanno quell'implicante (riga orizzontale, cancello tutte le colonne corrispondenti di cui incrocio le x su quella riga)
   3. Elimino la riga di quell'implicante



   (b) **Dominanza di Riga** (se una riga A ha tutte le X di un'atra, B, e anche di più, si dice che Adomina B e quindi posso eliminare B)



   (c) **Dominanza di colonna** (Se una colonna C ha le X di una colonna D, e D ne ha anche altre, C domina D, quindi posso eliminare D)



5. Riporto la copertura minima come somma degli implicanti essenziali e di quelli sopravvissuti alle varie eliminazioni

# gruppo	mintermini (codice binario)
0	m_0 : 0000
1	m_1, m_2, m_4, m_8 : 0001, 0010, 0100, 1000
2	$m_3, m_5, m_6, m_9, m_{10}, m_{12}$: 0011, 0101, 0110, 1001, 1010, 1100
3	$m_7, m_{11}, m_{13}, m_{14}$: 0111, 1011, 1101, 1110
4	m_{15} : 1111

Funzione di Output Multipla

1. Passaggi preliminari (Tabella 1 $\rightarrow$ 2 e Tabella 2 $\rightarrow$ 3):
2. **Tabella 1 $\rightarrow$ Tabella 2**:
   (a) **Divisioni preposte**: non riordinare i mintermini; utilizzare le colonne definite dal conteggio dei "1".
   (b) **Confronti 1-bit**: confronta solo gruppi adiacenti (colonna k vs $k+1$); unisci se e solo se:
   - l'implicante di partenza è numericamente minore di quello d'arrivo,
   - la differenza (arrivo $-$ partenza) è un multiplo positivo di 2,
   - i vettori differiscono in esattamente una posizione.

   (c) **Trucchetto di copiatura**: scrivi i bit del primo implicante, poi cancella leggermente il punto del nuovo "–" e aggiungilo a matita, per evitare errori di trascrizione.
   (d) **Marcatura e Pulizia**: marca come "usati" solo gli implicanti originali che partecipano a una fusione valida; elimina subito i composti con bit-appartenenza "00". Per essere valida, l'AND bit a bit delle maschere deve essere uguale

Maschera	Raccoglimento	Marcatura
$t_{ab} = 0 \dots 0$	non valido	nessuna
$t_{ab} = t_a$	valido	solo p_a
$t_{ab} = t_b$	valido	solo p_b
$t_{ab} = t_a = t_b$	valido	p_a e p_b

3. **Tabella 2 $\rightarrow$ Tabella 3**:
   (a) **Raggruppa**: in base al numero di "1" ancora espliciti (contando "–" come zero).
   (b) **Confronti –bit**: cerca DC (-) sulla stessa colonna, li evidenzio per essere facilitato visivamente.
   (c) **Trucchetto di copiatura**: stessa procedura di Tabella 1 $\rightarrow$ 2 per evitare errori.
   (d) **Marcatura e Duplicati**: marca come "usati" i genitori di ogni fusione anche se poi elimini i duplicati; rimuovi permutazioni identiche (es. 0,1,8,9 vs 0,8,1,9).
   (e) **Scarto**: ometti sempre i nuovi implicanti con bit-appartenenza "00".

Poi in questa tabella **NON RIPORTO DCSET!!**:

	Funz. 1 On-set			Funz. 2 On-set		
Implicante	0	1	2	2	3	9
A						
B						

1. In caso di più funzioni:
   (a) **Essenzialità: su singola tabella**
   (b) Dominanza di righe: **su entrambe le funzioni at once**
   (c) Dominanza di **colonna: su singola tabella**
2. Riporto la copertura minima come somma degli implicanti essenziali e di quelli sopravvissuti alle varie eliminazioni

*Per la **DOMINANZA DI RIGA, se il costo** della funzione dominante **è minore** di quello della dominata, allora posso effettuare l'eliminazione; **viceversa NON posso** cancellare la riga !*

Funzione di costo **numero di implicanti**: implicante essenziale diventa a COSTO **ZERO 0**

Funzione di costo **numero di letterali**: implicante essenziale diventa a COSTO **UNO 1**

Modello algebrico e reti multilivello

Calcolo costo

- Identifico tutte le uscite, per ogni uscita conto i letterali (forma SOP) e poi sommo i costi delle singole uscite

1. Calcoliamo il costo della rete iniziale

	espressione	**costo**
$p =$	$bc + bde' + a'c' + ade' + a'e$	12
$q =$	$dc' + b'ce + a'e + de + a'c'$	11
$r =$	$eb' + adb + c'b'$	7
$s =$	$a' + d$	2
	Costo(RETE)	32

Grafo associato alla rete logica

- Ingressi a sinistra, funzioni logiche in mezzo e uscite a destra. Collego ad ogni espressione gli ingressi presenti

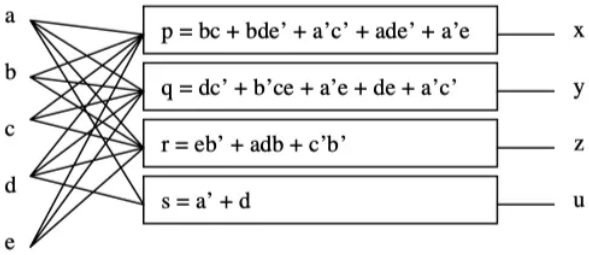
Esercizio 4.1 - Utilizzando il modello algebrico, si consideri la rete logica definita dalle seguenti espressioni:

$$\begin{aligned}
&\mathrm{p = bc + bdee' + a'c' + adee' + a'e}\\
&\mathrm{q = dc' + b'ce + a'e + de + a'c'}\\
&\mathrm{r = ebb' + adb + c'b'}\\
&\mathrm{s = a' + d}\\
&\mathrm{x = p}\\
&\mathrm{y = q}\\
&\mathrm{z = r}\\
&\mathrm{u = s}
\end{aligned}$$

dove $\mathrm{a, b, c, d,}$ e sono gli ingressi e $\mathrm{x, y, z, u}$ sono le uscite.

1. Si disegni il grafo associato alla rete logica e si calcoli il costo associato in termini di letterali Costo(RETE).

Il grafo associato alla rete iniziale è:



Trasformazioni della rete

Solitamente se il costo complessivo della rete **migliora** a seguito di una trasformazione, questa è da ritenersi effettiva, altrimenti no.

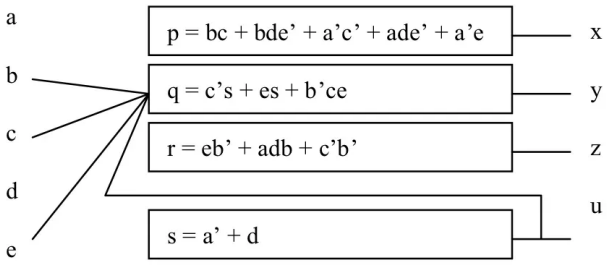
(a) Sostituire s in q.

Iniziamo con una fattorizzazione di q per cercare eventuali *occorrenze* di s

	espressione		**costo**
$q =$	$c'(d + a') + e(d + a') + b'ce$	$c'\underline{s} + e\underline{s} + b'ce$	7
	Costo(RETE)		$32 \Rightarrow 28$

Il Costo(q) = 7 dunque il Costo(RETE) = $28 < 32$ per cui accettiamo.

Il grafo associato alla rete trasformata è:



Si mostrano solo le connessioni relative a Q

Teorema di Espansione di Shannon

IMPORTANTE: se viene richiesto di cercare la forma canonica, procedo con Shannon iterativamente MA NON DEVO applicare semplificazioni algebriche!

Data una funzione $f(a, b, c)$ è possibile riscrivere la suddetta come somma di due sotto-funzioni che hanno i valori di una delle variabili fissati come segue:

$$f(a, b, c) = \overline{a} \cdot f(0, b, c) + a \cdot f(1, b, c)$$

Il procedimento può essere reiterato.

Esercizio 4.2 - Si consideri la seguente funzione

$$\begin{aligned}
f(a, b, c, d) =& a'b'cd' + a'b'cd + a'bc' + a'bc'd\\
&+ abcd' + abc'd + ab'cd' + ab'cd
\end{aligned}$$

decomporre f in maniera disgiuntiva rispetto alle variabili a, b e operando una trasformazione locale calcolarne la variazione di costo.

SOLUZIONE

Per il *teorema di espansione di Shannon* dobbiamo pervenire ad una funzione in questa forma

$$\begin{aligned}
f(a, b, c, d) =& a'b' \cdot f(0, 0, c, d) + a'b \cdot f(0, 1, c, d)\\
&+ ab' \cdot f(1, 0, c, d)\\
&+ ab \cdot f(1, 1, c, d)
\end{aligned}$$

il cui $\text{Costo}_{\text{prima}}(f) = 31$.

$$\begin{aligned}
f(0, 0, c, d) &= cd' + cd = c(d' + d) = c\\
f(0, 1, c, d) &= c' + c'd = c'\\
f(1, 0, c, d) &= cd' + cd = c(d' + d) = c\\
f(1, 1, c, d) &= cd' + c'd
\end{aligned}$$

Calcoliamo la nuova espressione associata al nodo e la variazione di costo

$$f(a, b, c, d) = a'b'c' + a'bc' + ab'c + abcd' + abc'd$$

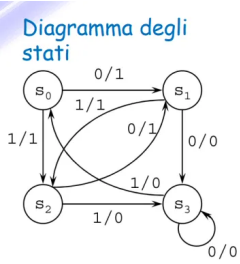
con $\text{Costo}_{\text{dopo}}(f) = 17$. Pertanto $\Delta\text{Costo} = 14$.

Minimizz. FSM completamente specificate

Completamente specificate → transizioni definite ∀ INPUT ∀ STATO

Macchina di Mealy (API-style)

Le uscite dipendono dalle transizioni, non dagli stati.



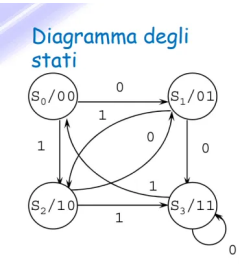
	0	1
S_0	$S_1/1$	$S_2/1$
S_1	$S_3/0$	$S_2/1$
S_2	$S_1/1$	$S_3/0$
S_3	$S_3/0$	$S_0/0$

Nella tabella è segnato come:

- Stato di arrivo/Valore di Uscita, prendendo in ingresso Stato e Valore di Input

Macchina di Moore

Il valore delle uscite dipende dallo stato in cui ci si trova.



	0	1	U
S_0	S_1	S_2	00
S_1	S_3	S_2	01
S_2	S_1	S_3	10
S_3	S_3	S_0	11

Nella tabella vengono segnati gli stati di arrivo e a parte l'uscita relativa allo stato di partenza

Insieme degli stati raggiungibili

Data la tabella degli stati procedo nel seguente modo:

1. A partire dallo stato di reset segno tutti i possibili stati raggiungibili in colonna
2. una volta scritti tutti gli stati raggiungibili dallo stato corrente, passo al secondo elemento della colonna, e scrivo tutti gli stati raggiungibili da quello stato (**solo se non ancora presenti**)
3. itero il procedimento finche non torno al punto di partenza o non ho terminato gli stati visitabili.
4. La colonna risultante è l'insieme degli stati raggiungibili

Esercizio 6.1 - Data ka tabella degli stati:

	0	1
Reset	G/00	G/00
B	G/00	D/01
C	−/−	Reset /11
D	C/10	D/11
E	G/00	F/11
F	F/10	E/10
G	Reset /01	F/11

quelli scritti a lato sono gli stati raggiungibili

*Si segue un procedimento analogo nel caso in cui la tabella degli stati **sia** quella **di una macchina di Moore***

Minimizzazione di un FSM - Metodo di Paul Unger

Sempre a partire dalla tabella degli stati, costruisco una matrice triangolare bassa che ha sulle colonne l'elenco degli stati **a partire dal secondo**, e sulle righe gli stati partendo dal primo (**escludendo l'ultimo**)

Esercizio 6.2 - Data la seguente tabella degli stati di una FSM sincrona di Moore, minimizzarla.

	0	1	OUT
A	A	C	0
B	F	A	0
C	D	E	1
D	C	E	1
E	B	F	1
F	F	D	0

B					
C					
D					
E					
F					
	A	B	C	D	E

Su colonna verticale elimino il primo stato, sull'orizzontale l'ultimo

Una volta impostata la tabella procedo allo riempimento:

1. Confronto l'uscita dello stato sulla riga con quella dello stato sulla colonna
   (a) se $OUT_1 = OUT_2 \implies$ confronto gli stati che raggiungerei in caso leggessi 0 o 1 da input
      i. se coincidono, segno nella casella della tabella in analisi $\sim$ (in questo caso **i due stati SONO equivalenti** fra loro)
      ii. se non coincidono, segno nella tabella i due stati, così da ricordarmi di verificare se sono equivalenti o meno
   (b) se $OUT_1 \neq OUT_2 \implies$ segno nella casella una X(in questo caso i due stati NON sono equivalenti fra loro)
2. itero il procedimento fino ad aver riempito completamente la tabella
3. Una volta terminati gli step 1 e 2, creo una nuova tabella in cui copio il contenuto delle caselle segnate con X e $\sim$
4. per le altre caselle vado a verificare se gli stati segnati sono equivalenti
   (a) se lo sono posso sostituire il contenuto con $\sim$
   (b) se non lo sono metto una X

   *In caso di **ciclicità**, si dice che i due stati in questione sono equivalenti fra loro*

   Costruendo la tabella (triangolare) delle implicazioni si ottiene:

   - primo passo

B	AF CA				
C	X	X			
D	X	X	$\sim$		
E	X	X	DB EF	CB EF	
F	CD	AD	X	X	X
	A	B	C	D	E

   - secondo passo

B	X				
C	X	X			
D	X	X	$\sim$		
E	X	X	X	X	
F	$\sim$	X	X	X	X
	A	B	C	D	E

5. Raggruppo gli stati equivalenti in uno che li contiene (si può fare sia "ad occhio" che con il grafo.)
   (a) *Per il grafo si scrivono tutti gli stati e si collegano quelli compatibili. Le figure piane che ne derivano rappresentano i gruppi di compatibilità, punti e linee comprese*
      i. Nel nostro caso: $\alpha = \{A, F\}$, $\beta = \{C, D\}$, B, E
6. Posso ricostruire la tabella degli stati ridotta

   *Nel caso di **macchina di Mealy**, il procedimento è lo stesso, ma la verifica sull'identicità dell'uscita la devo fare nelle singole celle e dato che non dipende più dallo stato, bensì dal valore di ingresso*

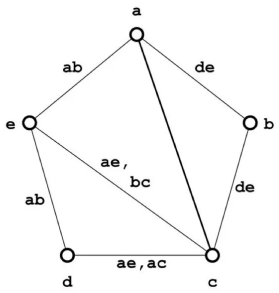
Minimizzazione di FSM NON completamente specificate

Il processo di minimizzazione è analogo al caso completamente specificato , solo che ora sono presenti anche i " $-$ " (don't care, DC). Li tratto alla solita maniera

- **Analisi di raggiungibilità da fare solo se richiesta espressamente**, specie con **FSM non completamente specificati**
- Se le uscite sono uguali e/o composte da $1/0$ e $- \implies$ posso guardare se gli stati sono equivalenti o meno
- Se le uscite sono differenti $\implies$ posso segnare direttamente X, anche se sono presenti DC

Nel caso di **FSM non completamente specificati**, **i cicli NON vengono considerati** come **equivalenti**, ovvero **le compatibilità NON si propagano da una cella all'altra, si lasciano i vincoli**, sia **nella semimatrice** che nel **grafo**. Una volta fatto il grafo delle compatibilità **scrivo i vincoli** di compatibilità **sugli archi** che collegano i due stati interessati. **L'incompatibilità si propaga normalmemnte invece!**

	a	b	c	d
b	de			
c	∨	de		
d	x	x	ae ac	
e	ab	x	ae bc	ab



Algoritmo ad Albero per minimizzazione FSM

1. **Grafo di compatibilità**
   (a) Prepara la **tabella triangolare** delle coppie di stati, con solo:
   - X per stati incompatibili,
   - $\sim$ per stati potenzialmente compatibili.

   (b) **Disegna un grafo** in cui:
   - Ogni **nodo** è uno stato della FSM.
   - **C'è un arco** tra due stati solo se nella tabella risultano compatibili.
2. **Nodo radice**
   - Crea **un unico nodo** (radice) con **tutti gli stati** della FSM all'interno.
3. **Procedura di "taglio"**
   (a) Tira riga diritta per taglio, andando in ordine, ad esempio S1
   (b) Crea **due nodi figli**:
   - **Nodo sinistro:** contiene **tutti** gli stati del nodo padre **tranne** lo stato su cui stai tagliando (es. S1).
   - **Nodo destro:** contiene
     i. Lo stato su cui stai tagliando (S1).
     ii. Tutti gli stati del nodo padre che hai già "tagliato" prima (se presenti).
     iii. Tutti gli stati (del nodo padre) **compatibili** con S1 (cioè collegati a S1 nel grafo).
     iv. Ripeti il "taglio" per i nuovi nodi (figli) se necessario.
4. **Nodi foglia**
   - Un **nodo** diventa **foglia** quando gli stati al suo interno **sono già stati tutti esaminati**, tranne **al più uno** (cioè non c'è più possibilità/necessità di ulteriori "tagli").
   - Se trovi **due foglie uguali** (stesso identico insieme di stati), **elimina** uno dei due (duplicato).
5. **Risultato**
   - Al termine, **i nodi foglia rimanenti** sono le **classi di massima compatibilità**.
   - Attenzione: **non è garantito** che l'FSM risultante sia effettivamente la **minima** possibile, perché il problema è NP-completo. Tuttavia, queste foglie rappresentano comunque gruppi di stati compatibili.

Common mistake: le **foglie** vanno **eliminate solo se** $\in$ **altre foglie**, sennò le si riquadra e ci si ferma. **Non é che se trovo una foglia allora la elimino subito!**

Verificare che rete sia implementazione di un FSM

- Numerare i flip-flop presenti nella rete (ad es. D_0/Q_0, T_1/Q_1, D_2/Q_2, ...).
- Calcolare il numero totale di stati da analizzare tramite la tabella di verità:

$$\text{righe} = 2^{\#\text{flip-flop}}$$

- Ricavare le equazioni di transizione e di uscita, ad esempio

$$D_0 = \ldots, \quad D_1 = \ldots, \quad Z = \ldots$$

- Per ciascun valore dell'ingresso X (cioè $X = 0$ e $X = 1$):
  – Creare la seguente tabella, con le righe che corrispondono agli stati correnti (Q_1Q_0), e le colonne dedicate a D_1, D_0 e Z per $X = 0$ e $X = 1$:

Stato	$X=0$			$X=1$		
Q_1Q_0	D_0	D_1	Z	D_0	D_1	Z
00						
01						
10						
11						

Algebra Binaria

Moltiplicazione Binaria

Si esegue come una normale moltiplicazione in colonna

Moltiplicazione con operandi negativi (in C_2)

Si svolge la solita moltiplicazione in colonna ma:

- Se il moltiplicatore **è negativo**, bisogna invertire il segno di moltiplicando e moltiplicatore
  - $P = A \cdot B$, se $B < 0 \implies P = (-A) \cdot (-B)$

    Sticazzi del nome, l'importante è che quello negativo sia il primo nell'operazione

```
00111 *    =  7_10
11000 =    = -8_10

     11001 *    = -7_10
     01000 =    =  8_10
  ----------
     00000 +
    00000  +
   00000   +
  11001    +
 00000     =
  ----------

     11001 *    = -7_10
     01000 =    =  8_10
  ------------
 0000000000 +
 000000000  +
 00000000   +
 1111001    +
 000000     =
 ------------
 1111001000      = -56_10
```

- Si estende il segno dei prodotti parziali, tanti bit fino a quanto il prodotto risulta grande(n+m bit)

```
     10011 *    = -13_10
     00101 =    = 5_10
  ----------
     10011 +
    00000  +
   10011   +
  00000    +
 00000     =
 ----------

     10011 *    = -13_10
     00101 =    = 5_10
  ------------
 1111110011 +
 0000000000 +
 1111110011 +
 0000000    +
 0000000    =
 ------------
 1110111111      = -65_10
```

Si ignora qualsiasi bit in posizione > n + m

Algoritmo di Booth

Radix-2

$$10 \times -5 = 01010_2 \times 11011_2$$

1. Entrambi i membri del prodotto devono essere in CA2
2. Aggiungo uno 0 in coda a LSB del mio moltiplicatore (a destra)

$$110110$$

3. Scompongo il numero in coppie di bit sovrapposte a partire da destra, estendendo il segno se necessario

$$11/10/01/11/10$$

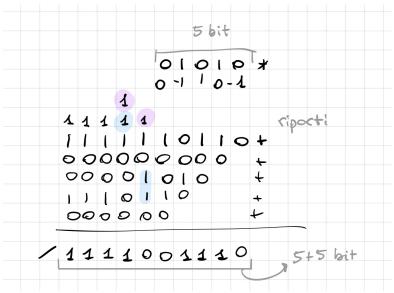
4. Applico le seguenti sostituzioni

Coppia moltiplicatore	Codifica
00	0
01	1
10	-2
11	0

$$11/10/01/11/10 \;=\; 0/-1/1/0/-1$$

5. Eseguo le moltiplicazioni nella nuova codifica
   (a) se nei prodotti parziali $* -1$, vuol dire che faccio il complemento a 2 del moltiplicando (10_{10}) ed **estendo il bit di segno** fino ad aver raggiunto l'$(n+m)$-esimo bit

```
          0 1 0 1 0 *  = 10_10
          0-1 1 0-1 =  = -5_10 (radix-2)
----------------------
 1 1 1 1 1 1 0 1 1 0 +
 0 0 0 0 0 0 0 0 0   +
 0 0 0 0 1 0 1 0     +
 1 1 1 0 1 1 0       +
 0 0 0 0 0 0
----------------------
 1 1 1 1 0 0 1 1 1 0     = -50_10
```



Radix-4

https://www.youtube.com/watch?v=SGnab30ISwg&ab_channel=AlwaysLearnMore

Come per radix-2, cambiano però il raccoglimento e la codifica:

1. Aggiungo uno 0 in coda a LSB del mio moltiplicatore (a destra)
2. Scompongo il numero in gruppi da 3 bit sovrapposti a partire da destra, estendendo il segno se necessario

$$00100110\underline{0} = 001 \,/\, 100 \,/\, 011 \,/\, 100$$

1. Applico le seguenti sostituzioni, diverse da radix2
   In Radix4 si raggruppa spostandosi di 2 bit in 2 bit, dopo aver aggiunto lo zero a destra. In Radix2, per raggruppare, ci si sposta di un bit alla volta

000	0
001	1
010	1
011	2
100	-2
101	-1
110	-1
111	0

$$00100110\underline{0} = 001 \,/\, 100 \,/\, 011 \,/\, 100$$
$$= 1 \quad -2 \quad 2 \quad -2$$

1. Eseguo le moltiplicazioni nella nuova codifica
   (a) se -2, vuol dire che faccio shift a sinistra dei bit,

```
                    -3_10 * 38_10
                 1111101_2 * 0100110_2
-------------------------------------------------
Moltiplicare per (-)2 significa shiftare i
bit a sinistra di 1 posizione
          1 1 1 1 1 0 1 *  = -3_10
          1-2 2-2 =  = 38_10 (radix-4)
--------------------------
0 0 0 0 0 0 0 0 0 1 1 0 +
1 1 1 1 1 1 1 1 0 1 0   +      Ricordarsi i due spazi in radix-4
0 0 0 0 0 0 1 1 0       +
1 1 1 1 1 0 1           +
--------------------------
1 1 1 1 1 1 0 0 0 1 1 1 0     = -114_10
```

Risultato di RADIX2/4 in CA2. Per passare da CA2 a decimale, se bit di segno (MSB) é 0 (+), allora lo interpreto come un normale binario.
Se invece bit di segno é 1 (-), allora faccio CA1 (inversione bit), e +1, riportando in decimale ottengo il modulo. Devo chiaramente ricordarmi del $-$ del bit di segno.

Virgola fissa

https://www.youtube.com/watch?v=tzmihLszgNY

Virgola Mobile – IEEE 754 Single Precision (32 bit)

Un'unica procedura chiara in 5 passi, con note sulle difficoltà più comuni:

1. **Bit di segno (1 bit)**:
   - 0 se il numero è positivo
   - 1 se il numero è negativo
2. **Rappresentazione raw (denormalizzata)**:
   (a) *Parte intera*: converti N_{int} in binario $(b_k \dots b_0)_2$.
   (b) *Parte frazionaria*: moltiplica f per 2 ripetutamente:
   $$f \times 2 \;\rightarrow\; \text{bit} + \text{resto}.$$
   Tieni una tabella $\{\text{resto} \rightarrow \text{posizione}\}$. Appena
   - trovi un resto già visto (hai individuato il ciclo) *e*
   - hai generato almeno 24 bit (23 mantissa + 1 bit extra per arrotondare),

   ti fermi: avrai identificato il pattern ciclico (p. es. "0011") ed eviterai moltiplicazioni infinite.

   Scrivi quindi:
   $$N_{10} = (\text{intera})_2.(\text{frazione})_2.$$
3. **Normalizzazione e calcolo dell'esponente vero** E:
   - Sposta il punto binario fino ad avere un solo "1" a sinistra:
     $$100111010.0001\ldots_2 = 1.001110100001\ldots_2 \times 2^n.$$
   - Il numero di spostamenti n dipende *solo* dai bit della parte intera.
   - L'esponente vero è $E = n$; **biased**: $E_b = E + 127$.
4. **Estrai mantissa (23 bit)**:
   - Dopo la normalizzazione, prendi i primi 23 bit subito dopo il punto.
   - Se la frazione è ciclica, continua a ripetere il pattern finché hai quei 23 bit
5. **Arrotondamento e assemblaggio finale**:
   (a) Applica la regola *round-to-nearest even* usando il bit extra per decidere se arrotondare su o troncare.
   (b) Tronca al campo fraction (23 bit).
   (c) Disponi i 32 bit come
   $$\underbrace{s}_{1} \;|\; \underbrace{e_7 e_6 \dots e_0}_{8} \;|\; \underbrace{f_1 f_2 \dots f_{23}}_{23}.$$
6. **Esempio rapido (12,075)**:
   (a) Segno: 0
   (b) Raw: $12_{10} = 1100_2$, fraz=$0.075 \rightarrow 0.0001\,0011\ldots$ (ciclo=0011)
   (c) Normalizza:
   $$1100.0001\ldots = 1.1000001\ldots \times 2^3, \quad E = 3, \quad E_b = 130 = 10000010_2.$$
   (d) Mantissa: primi 23 bit `00010011001100110011001`, bit extra per arrotondare = prossimo "1"
   (e) Arrotonda $\rightarrow$ mantissa finale: `00010011001100110011010`
   (f) IEEE 754:
   $$0 \mid 10000010 \mid 00010011001100110011010 = \texttt{0x41266CCD}.$$

Convertire a base 10

$$\text{Valore} \;=\; (-1)^s \times 2^{E_{\text{reale}}} \times M$$

- **Bit di segno** s: 0 = positivo, 1 = negativo.
- **Esponente codificato** e:
$$E_{\text{reale}} = \begin{cases} \text{-126}, & e = 00000000 \text{ (\textbf{denormalizzato})} \\ e - 127, & 00000001 \le e \le 11111110 \text{ (\textbf{normalizzato})} \\ \textbf{speciale}, & e = 11111111 \end{cases}$$
- **Mantissa** M:
$$M = \begin{cases} (\mathbf{0}.f_1 f_2 \dots f_{23})_2, & e = 00000000 \\ (\mathbf{1}.f_1 f_2 \dots f_{23})_2, & \text{altrimenti} \end{cases}$$

Nota sul bit implicito della mantissa
In *normalizzati* ($e \neq 00000000$):

$$M = (1.f_1 f_2 \dots f_{23})_2 \quad \text{il "1." è \textbf{implicito}, non memorizzato.}$$

In *denormalizzati* ($e = 00000000$):

$$M = (0.f_1 f_2 \dots f_{23})_2 \quad \text{lo "0." è \textbf{esplicito}.}$$

Esempio Normalizzato ($+5.0$)

$$0 \mid 10000001 \mid 01000000000000000000000$$

- $s = 0 \rightarrow$ positivo.
- $e = 10000001_2 = 129 \rightarrow E = 129 - 127 = 2$.
- $f = 010\ldots0 \rightarrow M = 1.01_2 = 1.25$.
- Valore $= +1.25 \times 2^2 = 5.0$.

Esempio Denormalizzato

$$0 \mid 00000000 \mid 00000000000000000000001$$

- $s = 0 \rightarrow$ positivo.
- $e = 00000000 \rightarrow E = -126$.
- $f = \ldots001 \rightarrow M = 0.000\ldots01_2 = 2^{-23}$.
- Valore $= +2^{-126} \times 2^{-23} = 2^{-149}$.

Aritmetica in IEEE 754 Single Precision

1. **Procedura unificata per addizione e sottrazione**
   (a) *Forma normalizzata:* esprimiamo i due operandi come
   $$A = (-1)^{S_A}\ 1.M_A \times 2^{E_A}, \quad B = (-1)^{S_B}\ 1.M_B \times 2^{E_B}.$$
   (b) *Allineamento degli esponenti:* poniamo $E = \max(E_A, E_B)$. Per ogni incremento di esponente spostiamo di un bit a destra la mantissa del numero con esponente minore, finché entrambi gli esponenti diventano E. L'esponente comune E (quello del numero di ordine maggiore) non sarà più modificato.
   (c) *Operazione sulle mantisse:* Eseguiamo
   $$(-1)^{S_A}\ M'_A \;\pm\; (-1)^{S_B}\ M'_B,$$
   dove M'_A, M'_B sono le mantisse già shiftate. Solo la mantissa cambia; l'esponente rimane E. Il segno del risultato, nel caso di sottrazione, sarà quello dell'operando con mantissa di valore assoluto maggiore.
   (d) *Normalizzazione:* Se la mantissa risultante non è in $[1.0, 2.0)$, la riallineiamo spostando il punto e aggiornando E di conseguenza ($+k$ o $-k$).
   (e) *Troncamento e codifica:* Tronchiamo la frazione a 23 bit (nessun arrotondamento), calcoliamo l'esponente biased $E_{\text{res}} = E + 127$, manteniamo il segno S_{res} e la mantissa normalizzata, quindi ricomponiamo il formato `S | E | M`.
2. **Esempi numerici**
   (a) *Addizione:* $1.5 + 0.5 = 2.0$
   - $1.5 = 0 \mid 01111111 \mid 10000000000000000000000$ ($+1.1 \times 2^0$).
   - $0.5 = 0 \mid 01111110 \mid 00000000000000000000000$ ($+1.0 \times 2^{-1}$).
   - Allineamento a $E = 0$: shift di 1 bit su $1.0 \rightarrow 0.1$.
   - Somma mantisse: $1.1_2 + 0.1_2 = 10.0_2$.
   - Normalizzazione: $10.0_2 = 1.0_2 \times 2^1$, quindi $E \leftarrow 0 + 1 = 1$.
   - Troncamento: mantissa $1.000\ldots0$, esponente biased $127 + 1 = 128$.
   - Risultato: $2.0 = 0 \mid 10000000 \mid 00000000000000000000000$.

   (b) *Sottrazione:* $1.5 - 2.5 = -1.0$
   - $1.5 = 0 \mid 01111111 \mid 10000000000000000000000$ ($+1.1 \times 2^0$).
   - $2.5 = 0 \mid 10000000 \mid 01000000000000000000000$ ($+1.01 \times 2^1$).
   - Allineamento a $E = 1$: shift di 1 bit su $1.1 \rightarrow 0.11$.
   - Sottrazione mantisse: $0.11_2 - 1.01_2 = -0.10_2$ (mantissa $|0.10_2| = 0.5$, segno "−").
   - Normalizzazione: $-0.10_2 \times 2^1 = -1.0_2 \times 2^0$, quindi $E \leftarrow 1 - 1 = 0$.
   - Troncamento: mantissa $1.000\ldots0$, esponente biased $127 + 0 = 127$, segno 1.
   - Risultato: $-1.0 = 1 \mid 01111111 \mid 00000000000000000000000$.

Conversione Single $\rightarrow$ Double Precision

- **Contesto:** Single usa 8 bit di esponente (E_s), double ne usa 11 (E_d).
- *Segno:* rimane lo stesso bit s.
- *Esponente:*
  $$E_{\text{vero}} = e_s - 127, \quad e_d = E_{\text{vero}} + 1023.$$
  In parole: togli il bias single e aggiungi il bias double.
- *Mantissa:* Copia i 23 bit di frazione da single e aggiungi 29 zeri a destra per arrivare a 52 bit.
- *Risultato:* Hai lo stesso valore matematico, ma con precisione estesa.

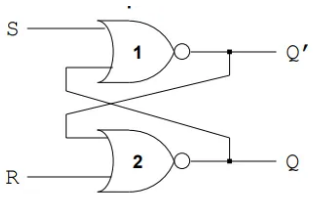
Table 1: Tabella dei Casi Speciali di IEEE 754 (Single Precision)

Esponente	**Mantissa**	**Significato**	**Conversione in base 10**
00000000	0	Zero (± 0)	Risultato: $(-1)^{\text{segno}} \times 0$
00000000	$\neq$ 0	Denormalizzato	$(-1)^{\text{segno}} \times 2^{-126} \times$ mantissa
11111111	0	$\pm$ Infinito	Risultato: $\pm\infty$
11111111	$\neq$ 0	NaN (Not a Number)	Nessuna conversione. Il risultato è "Non un Numero"
Altro (normale)	Qualsiasi	Normalizzato	$(-1)^{\text{segno}} \times 2^{\text{esponente reale}} \times (1 +$ mantissa $)$

Bistabili e Flip-Flop

Tabella **transizioni**: dato ingresso attuale Q_1, dice quale sarà il prossimo stato Q^*
Tabella **eccitazioni**: dato stato attuale Q, dice quali ingressi devo applicare per arrivare allo stato Q^*

Bistabile asincrono SR



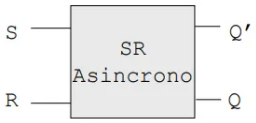


Tabella delle transizioni

Q \ SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

$\Longleftrightarrow$

S	R	Q*
0	0	Q
0	1	0
1	0	1
1	1	-

Tabella delle eccitazioni

Q	Q*	S	R
0	0	0	-
0	1	1	0
1	0	0	1
1	1	-	0

Espressione logica

$Q^* = S + R'Q$

Con vincolo $S = R \neq 1$

$Q*$:	stato futuro
Q :	stato presente

Diagramma Temporale

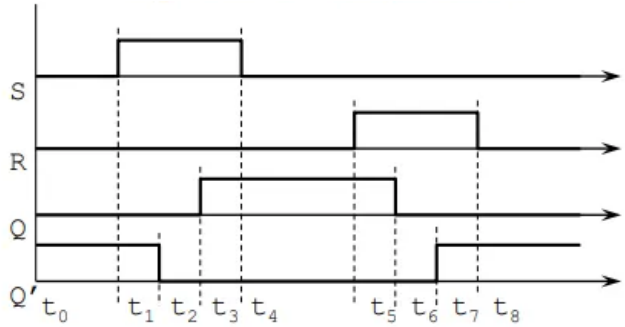


Tabelle di Transizioni ed Eccitazioni dei vari FF

Tabelle delle Transizioni:

S	R	Q*
0	0	Q
0	1	0
1	0	1
1	1	-

J	K	Q*
0	0	Q
0	1	0
1	0	1
1	1	Q*

D	Q*
0	0
1	1

T	Q*
0	Q
1	Q'

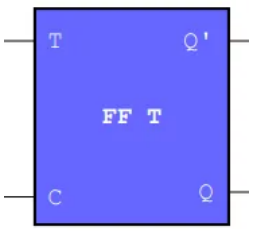
Tabelle delle Eccitazioni:

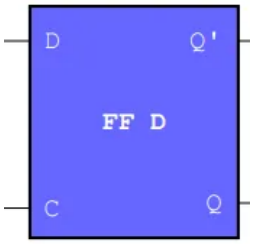
Q	Q*	S	R
0	0	0	-
0	1	1	0
1	0	0	1
1	1	-	0

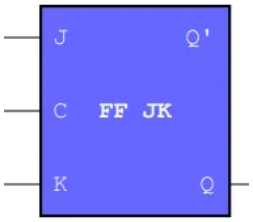
Q	Q*	J	K
0	0	0	-
0	1	1	0
1	0	-	1
1	1	-	0

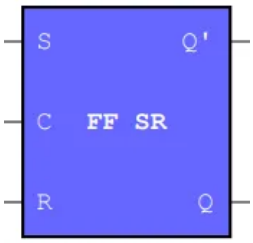
Q	Q*	D
0	0	0
0	1	1
1	0	0
1	1	1

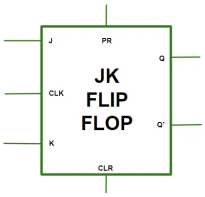
Q	Q*	T
0	0	0
0	1	1
1	0	1
1	1	0



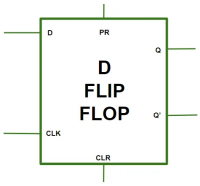


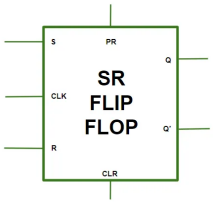












Porte logiche

MUX

$$F = SD_1 + \overline{S}D_0$$

- se $S = 1$ passa
- se $S = 0$ passa

(b) Sintetizzare $G(X, Y, Z) = X'Y + Z$ facendo uso di soli multiplexer a due ingressi. Si disegni

dato MUX: $F = SD_1 + \overline{S}D_0$

pseudo X come

$\rightarrow$ $D_0 = 4$ Se $X = 0$
$D_1 = 0$ Se $X = 1$

$\Rightarrow F = X \cdot 0 + \overline{X}Y = \overline{X}y$

Half Adder

Osservabilità e Controllabilità

F_{MAX} **– Funzione completa**
Calcola la tabella di verità dell'uscita assumendo di conoscere tutti i segnali (interni ed esterni), *senza* alcun vincolo di osservabilità o controllabilità. In pratica:

a) Prendi il circuito e identifica la funzione booleana F espressa sui segnali primari ed interni.

b) Costruisci la tabella completa di verità per tutti i segnali (ingressi e nodi interni).

c) Estrai la colonna corrispondente all'uscita: questa è F_{MAX}.

F_{OSS} **– Criterio di osservabilità**
Determina, per ogni combinazione degli ingressi, se il valore del *segnale interno* influisce sull'uscita:

a) Identifica il nodo interno da studiare (spesso indicato nel testo).

b) Per ciascuna riga della tabella di F_{MAX}, sostituisci nella funzione d'uscita il nodo interno con "0" e con "1":

$$\begin{cases} F_{\mathrm{out}}(n=0) \\ F_{\mathrm{out}}(n=1) \end{cases}$$

c) Se i due risultati coincidono, il nodo interno non è osservabile in quella riga: metti "–" in F_{OSS}.

d) Altrimenti, copia il valore da F_{MAX} in F_{OSS}.

F_{CON} **– Criterio di controllabilità**
Verifica quali combinazioni di ingressi *possono* portare il nodo interno ai valori 0 o 1:

a) Scrivi la tabella di verità del nodo interno $n = F(X, Y, Z)$ in funzione degli ingressi primari (A, B, C).

b) Per ogni tupla (X, Y, Z), trova tutte le righe di (A, B, C) che la generano.

c) Se non esiste alcuna riga che produca, per esempio, $X = 0$, allora in F_{CON} alla riga corrispondente a $(X = 0)$ metti "–".

d) Se esiste, copia da F_{MAX} il valore dell'uscita per quella combinazione.

F_{GLOB} **– Combinazione finale**
Integra osservabilità e controllabilità per stabilire la funzionalità effettiva:

$$F_{\mathrm{GLOB}}(r) = \begin{cases} -, & \text{se in riga } r \; F_{\mathrm{OSS}}(r) = - \;\vee\; F_{\mathrm{CON}}(r) = -, \\ F_{\mathrm{OSS}}(r) \;\cdot\; F_{\mathrm{CON}}(r), & \text{altrimenti.} \end{cases}$$

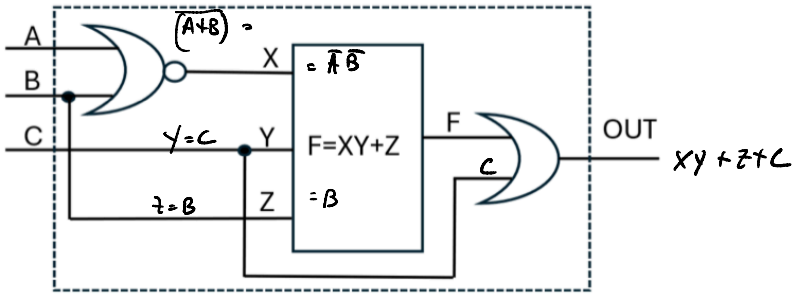
In altre parole:

- Se uno solo dei due criteri dà don't care, l'uscita resta "–".
- Se entrambi sono definiti, moltiplichi (in senso booleano) per ottenere il valore globale.

Oss. e Contr. esempio

ESERCIZIO 2

Sia dato il seguente circuito con tre ingressi primari da un bit (A, B e C) ed una uscita OUT, sempre da un bit. Si vuole progettare la funzione $F(X, Y, Z) = XY + Z$, che è interna al circuito, sfruttando le condizioni di **osservabilità** e di **controllabilità** per ridurne la complessità realizzativa.



- Si richiede di compilare la tabella sottostante ove sono già indicati in modo ordinato i possibili ingressi della F, ovvero X, Y e Z. Le colonne della tabella corrispondono ai seguenti casi:
  - F_{max} caso in cui **non** si applichi alcun criterio di osservabilità-controllabilità;
  - F_{oss} caso in cui si applichi la sola analisi di **osservabilità**;
  - F_{con} caso in cui si applichi la sola analisi di **controllabilità**;
  - F_{glob} caso in cui si applichi l'analisi congiunta di **osservabilità e controllabilità**.
- Si richiede infine, con riferimento alla sola F_{glob}, di ricavare la sua espressione minima facendo uso delle mappe di Karnaugh, disegnando il circuito equivalente di $F(X, Y, Z)$.

Per garantire la validità delle risposte, è necessario mostrare i passaggi fatti.

MAX; NO RESTRIZ.

X	Y	Z	F_{max}	F_{oss}	F_{con}	F_{glob}
0	0	0	0	0	0	0
0	0	1	1	1	1	1
0	1	0	0	–	0	–
0	1	1	1	–	1	–
1	0	0	0	0	0	0
1	0	1	1	1	–	–
1	1	0	1	–	1	–
1	1	1	1	–	–	–

A	B	C	X	Y	Z
0	0	0	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	1	0
1	1	0	0	0	1
1	1	1	0	1	1

CONTROLLO → IN F_{CON} SCELTO CORRISPETTIVO F_{MAX}. SE NON CONTROLLO, DON'T CARE

$f = XY + Z$

- F_{OSS}: VALORE INTERNO NON È OSSERVABILE (= DON'T CARE IN TABELLA F_{OSS}) SE ESSO NON SI PROPAGA FINO A USCITA.

→ IDENTICO SEGNALE INTERNO → F (dedotto dal testo)

Poiché OUT = F + C, se C = 1 non sarà mai osservabile

11

Y = 1

Oss. e Contr. esempio

- f_{CON}

  Controllabilità: - Cerco relazione da circ. che leg. var. ingressi di $f(X,Y,Z)$ a ingressi

$$\begin{cases} X = \bar{A}\,\bar{B} \\ Y = C \\ Z = B \end{cases}$$

A	B	C	X	Y	Z
0	0	0	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	1	0
1	1	0	0	0	1
1	1	1	0	1	1

→ TUTE QUESTE CONF. SONO CONTROLLABILI

- f_{OSS} → GUARDO f_{OSS} e f_{CON}
  - se c'è $(-)$, nella $(-)$
  - Sono AND $f_{OSS} \wedge f_{CON}$

Albero FSM non compl.spec.

ESERCIZIO 5

Data la seguente tabella degli stati di una macchina non completamente specificata con un ingresso X e una uscita Z. Si consideri lo stato A come stato di Reset.

S	X=0	X=1
A(RST)	D,1	C,-
B	A,-	D,0
C	B,0	A,1
D	-,1	F,-
E	-,0	-,1
F	D,1	E,-

(a) Si esegua l'analisi di compatibiltà usando il metodo di Paull-Unger.

(b) Si identificando le classi di massima compatibilità usando l'algoritmo dell'albero visto a lezione.

(c) Si riscriva la tabella degli stati ridotta usando le classi di massima compatibilità;

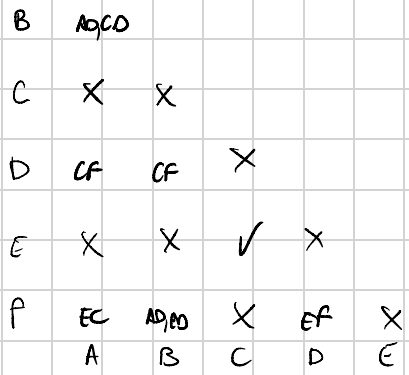
(d) Si scriva la tabella delle transizioni della macchina usando una codifica degli stati a numero minimo di FF.

(e) In quale modo l'assegnamento delle codifiche degli stati impatta la sintesi di una FSM?

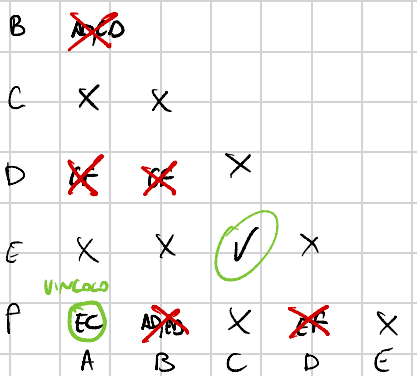
È necessario mostrare i passaggi fatti in ogni punto.

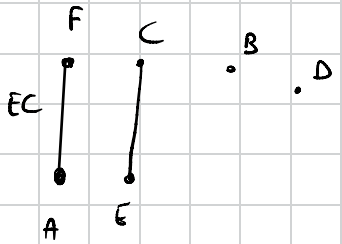
Sol.

FSM NON COMPLETAMENTE SPECIFICATA → NO PROPAGATION COMPATIBILITÀ (INCOMP. SI PROPAGA)

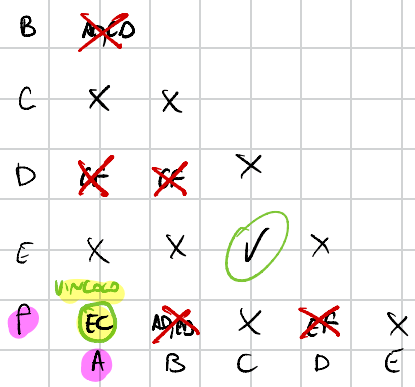


S	X=0	X=1
A(RST)	D,1	C,-
B	A,-	D,0
C	B,0	A,1
D	-,1	F,-
E	-,0	-,1
F	D,1	E,-

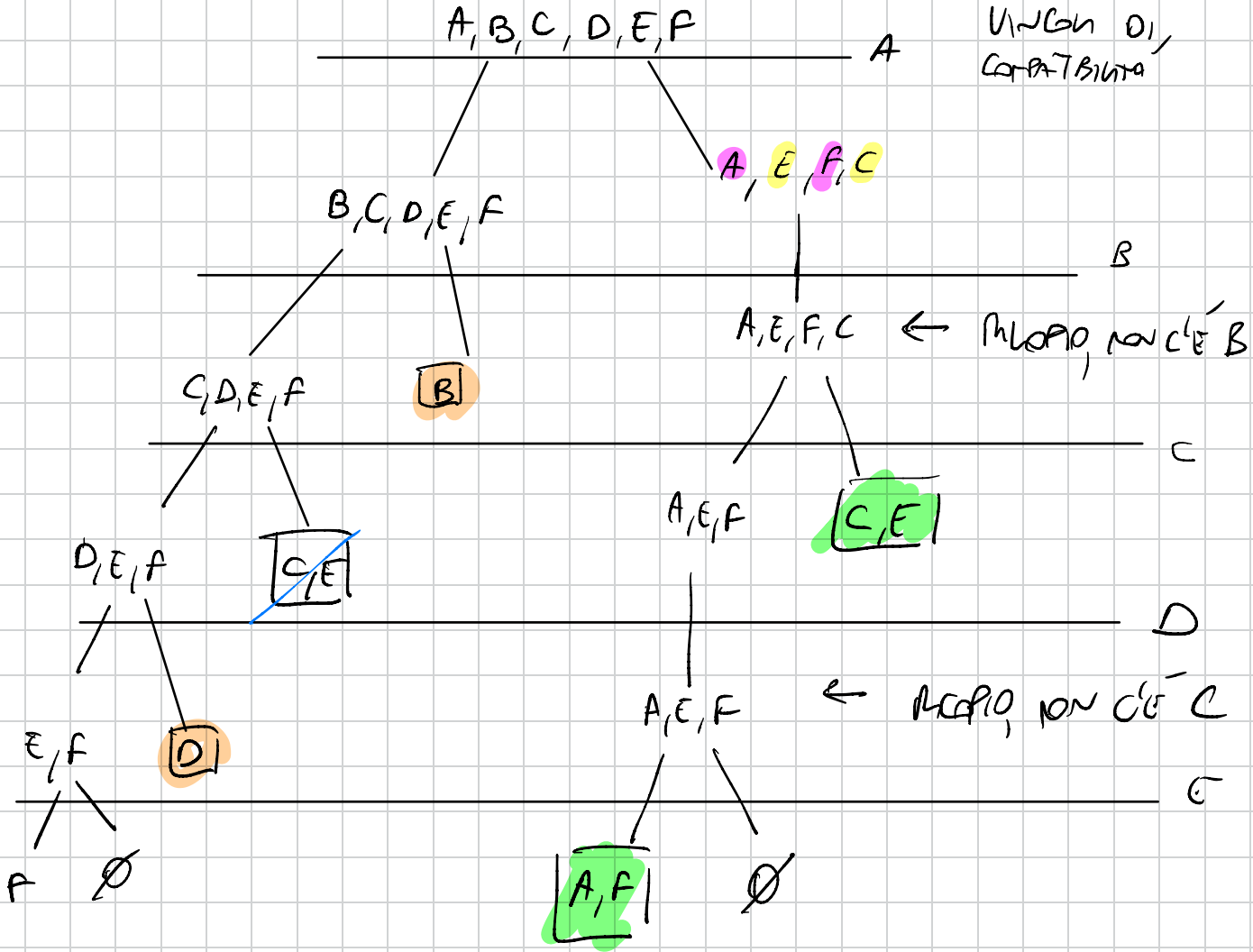




Albero FSM non compl.spec.



CON FSM NON SPECIFICATE METTO SEMPRE VINCOLI DI COMPATIBILITÀ



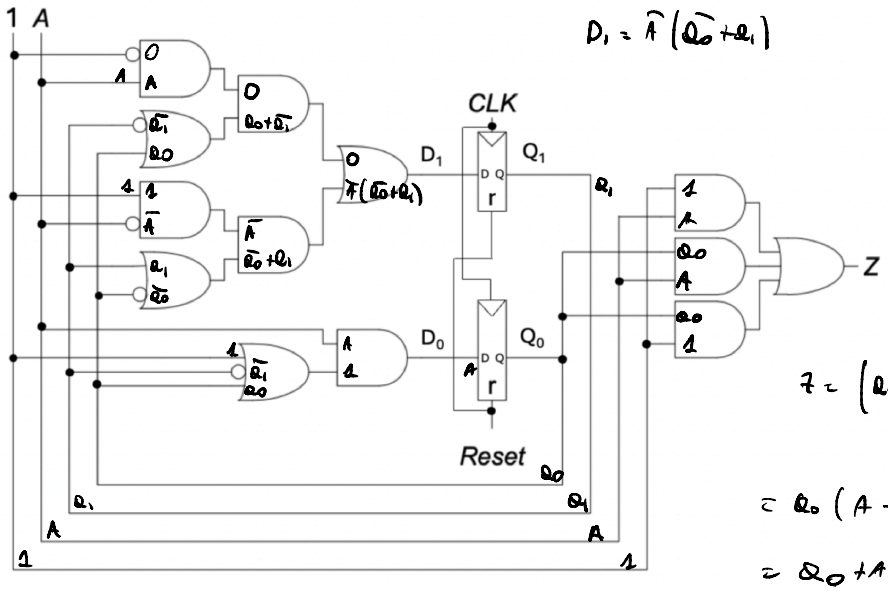
Da Rete a FSM - 1

ESERCIZIO 4

Data il seguente circuito rappresentante una macchina a stati completamente specificata con un ingresso A e una uscita Z

$D_0 = A$

$D_1 = \bar{A}(\bar{Q_0} + Q_1)$



$Z = (Q_0 A + Q_0 \bar{A}) =$

$= Q_0 (A + \bar{1}) + A =$

$= Q_0 + A$

(a) La macchina è di MEALY o di MOORE? Motivare la risposta;

(b) Si derivi la tabella degli stati;

(c) Partendo dal risultato ottenuto al punto precedente, si applichi l'analisi di raggiungibilità, rimuovendo gli stati irraggiungibili. Si consideri lo stato di reset pari a Q1=Q0=0 .

(d) Partendo dal risultato ottenuto al punto precedente, si applichi l'analisi di equivalenza con il metodo di Paull-Unger;

(e) Si identifichi la macchina minima equivalente e si riscriva la tabella degli stati ridotta;

Per garantire la validità delle risposte, è necessario mostrare i passaggi fatti.

Sol.

USCITA $Z = Q_0 + A$ (Q_0: STATO, A: INGRESSO) $\Rightarrow$ USCITA DIPENDE DA INGRESSO

$\downarrow$

MACCHINA DI MEALY

Da Rete a FSM - 1

$D_0 = A$

$D_1 = \bar{A}(\bar{Q_0} + Q_1)$

$z = Q_0 + A$

2 FF → $2^{\#FF}$ = 4 Stati = 2^2 FF-D

A = 0

	Q_1 Q_0	Q_1^* Q_0^*	z
S_0	0 0	(S_2) 1 0	0
S_1	0 1	(S_0) 0 0	1
S_2	1 0	(S_2) 1 0	0
S_3	1 1	(S_2) 1 0	1

A = 1

	Q_1 Q_0	Q_1^* Q_0^*	z
		(S_1) 0 1	1
		(S_1) 0 1	1
		(S_1) 0 1	1
		(S_1) 0 1	1

PER COMPLETARE:

S_0: "Da Stato S_0 = Q_1 Q_0 = 0 0, dove vado con A = 0/A = 1?

A = 0 ⇒ $Q_0^* = D_0 = A = 0$

$Q_1^* = D_1 = \bar{A}(\bar{Q_0} + Q_1) =$ $z = Q_0 \cdot A$

"VECCHIO VALORE"

A = 1 ⇒ $Q_0^* = D_0 = A$

⇒ Si nota che S_3 è irraggiungibile ⇒ lo rimuovo

Da Rete a FSM - 1

A = 0

	Q_1 Q_0	Q_1^+ Q_0^+	z
S_0	0 0	(S_2) 1 0	0
S_1	0 1	(S_0) 0 0	1
S_2	1 0	(S_2) 1 0	0
S_3	1 1	(S_2) 1 0	1

A = 1

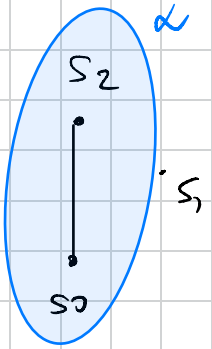
	Q_1^+ Q_0^+	z
(S_1)	0 1	1
(S_1)	0 1	1
(S_1)	0 1	1
(S_1)	0 1	1

	A = 0	A = 1
S_0	S_2 / 0	S_1 / 1
S_1	S_0 / 1	S_1 / 1
S_2	S_2 / 0	S_1 / 1

MACCHINA COMP. SPECIFICATA → ANALISI DI COMPATIBILITÀ

S_1	X		S_1	X		
S_2	S_2	X	S_2	✓	X	
	S_0	S_1		S_0	S_1	

⇒



- Scrivo stati delle righe formando le partizioni
- In colonna scrivo dal secondo

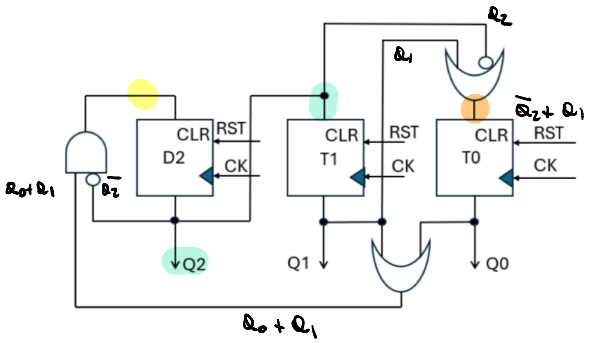
- TAB STATI RIDOTTA:

	A = 0	A = 1
α	α / 0	S_1 / 1
S_1	α / 1	S_1 / 1

Da Rete a FSM - 2

ESERCIZIO 4

Sia dato lo schema circuitale di un contatore a modulo e codifica arbitrari che fa uso di tre bistabili. Si noti che i bistabili sono di tipo diverso, nello specifico uno di tipo D e i due rimanenti di tipo T.



- Si ricavino le espressioni algebriche delle funzioni di eccitazione dei bistabili D_2, T_1 e T_0;
  - $D_2(Q_2, Q_1, Q_0) = \overline{Q_2} \cdot (Q_0 + Q_1)$
  - $T_1(Q_2, Q_1, Q_0) = Q_2$
  - $T_0(Q_2, Q_1, Q_0) = Q_1 + \overline{Q_2}$
- Assumendo che lo stato di reset del contatore sia quello rappresentato da $Q_2 = Q_1 = Q_0 = 0$, si ricavi il ciclo di conteggio compilando opportunamente la tabella sottostante. Lo si riporti in maniera esplicita anche negli spazi opportuni.

TRANSITION

Q_2	Q_1	Q_0	D_2	T_1	T_0	Q_2^*	Q_1^*	Q_0^*
0	0	0	0	0	1	0	0	1
0	0	1	1	0	1	1	0	0
1	0	0	0	1	0	0	1	0
0	1	0	1	0	1	1	1	1
1	1	1	0	1	1	0	0	0
0	0	0	...	...	...	...	...	...

CICLO DI CONTEGGIO (Q_2, Q_1, Q_0):

$000 \Rightarrow 001 \Rightarrow 100 \Rightarrow 010 \Rightarrow 111 \Rightarrow \ldots\ldots \Rightarrow \ldots\ldots \Rightarrow \ldots\ldots \Rightarrow \ldots\ldots$

- È come tracciare il ciclo di una macchina a stati finiti: inizi da 000, calcoli il prossimo stato ed arrivi a 001; da 001 calcoli il prossimo ed arrivi a 100; da 100 calcoli il prossimo ed arrivi a 010, e così via. Non sei obbligato a riempire le righe "statiche" nel solito ordine: segui invece la transizione **reattiva** data dalle equazioni.

Ragionato per un paio di secondi

Immagina il T-flip-flop come un interruttore di inversione:

- Se $T = 0$, non fa nulla e lascia il suo valore di Q com'era (resta "inattivo").
- Se $T = 1$, "togglia" (inverte) il bit: se era 0 diventa 1, se era 1 diventa 0.

Ora, per la prima parte (Q_1):

- Lo stato corrente di Q_1 è 0.
- Il segnale T_1 vale 0.

Dal punto di vista pratico:

- "$T_1 = 0$" significa "non toccare Q_1": l'interruttore di inversione è disinserito.
- Quindi Q_1 resta esattamente quello che era, cioè 0.

Perciò Q_1 *(il nuovo valore) rimane 0. In simboli:*

$Q_1^+ Q_1 \oplus T_1 = 0 \oplus 0 = 0,$

ma la spiegazione a parole è semplicemente "T è 0 → non cambio → rimango 0".

Da Tabelle a Rete

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

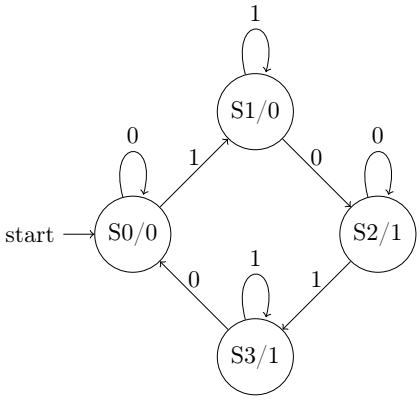
Esercizi Reti Logiche
Federico Reghenzani

Macchine a stati finiti - Finite State Machines (FSM)

---

1. Esempio 1 - Moore con FF D

Esercizio. Tradurre la seguente macchina a stati di Moore in VHDL e sintetizzarla tramite Flip Flop D:



con:

- $S = \{S0, S1, S2, S3\}$
- $I = \{0, 1\}$
- $U = \{0, 1\}$
- $\delta : S \times I \to S$
- $\lambda : S \to U$ (Moore)

Soluzione - VHDL. Vedi file su Webeep.

Soluzione - Sintesi. Scriviamo la tabella degli stati equivalente al diagramma della FSM:

S	$I = 0$	$I = 1$	U
S0	S0	S1	0
S1	S2	S1	0
S2	S2	S3	1
S3	S0	S3	1

Da Tabelle a Rete

Essendo 4 stati, possiamo codificarli (utilizzanndo la codifica naturale) attraverso 2 bit ($\lceil \log_2(4) \rceil = 2$). Convertiamo ora la tabella degli stati nella tabella delle transizioni:

Q_0Q_1	$I=0$	$I=1$	U
00	00	01	0
01	11	01	0
11	11	10	1
10	00	10	1

Sono necessari due Flip Flop e il testo dell'esercizio richiedeva l'uso di Flip Flop di tipo D.

Recap Flip Flop D.

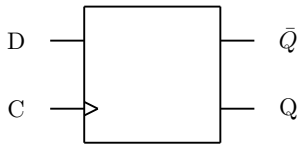


Tabella delle transizioni:

C	D	Q^*
0	-	Q
1	0	0
1	1	1

Tabella delle eccitazioni (Q: stato corrente, Q^*: stato successivo):

Q	Q^*	D	C
0	0	0	1
0	1	1	1
1	1	1	1
1	0	0	1
0	0	-	0
1	1	-	0

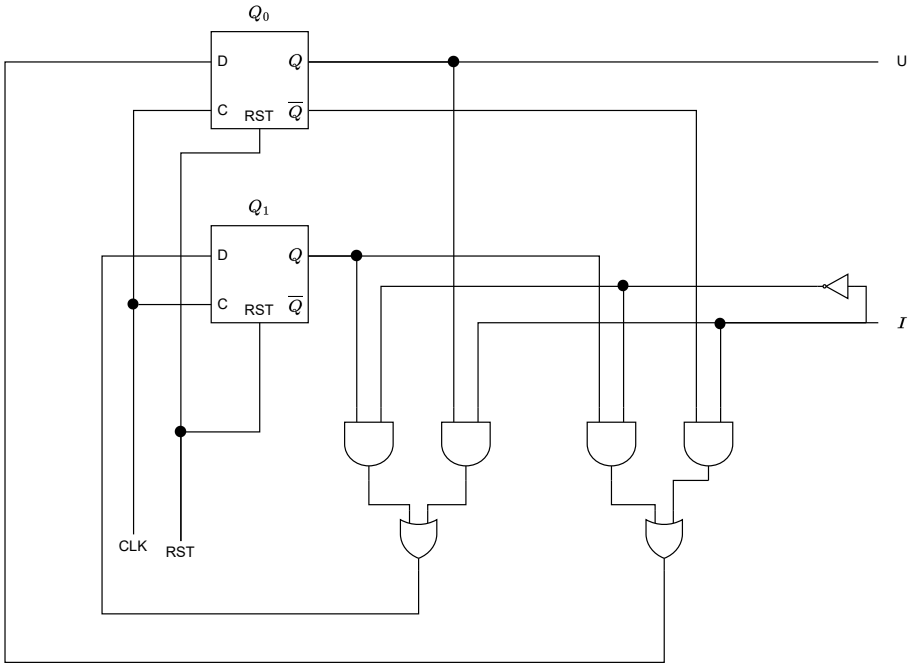
Procediamo ora con la sintesi tramite mappe di Karnaugh. Iniziamo sintetizzando la formula logica dell'ingresso D del primo flip-flop:

Q_0Q_1	$I=0$	$I=1$
00	0	0
01	1	0
11	1	1
10	0	1

e otteniamo: $D0 = Q_1\bar{I} + Q_0 I$.
Proseguiamo ora sintetizzando la formula logica dell'ingresso D del secondo flip-flop:

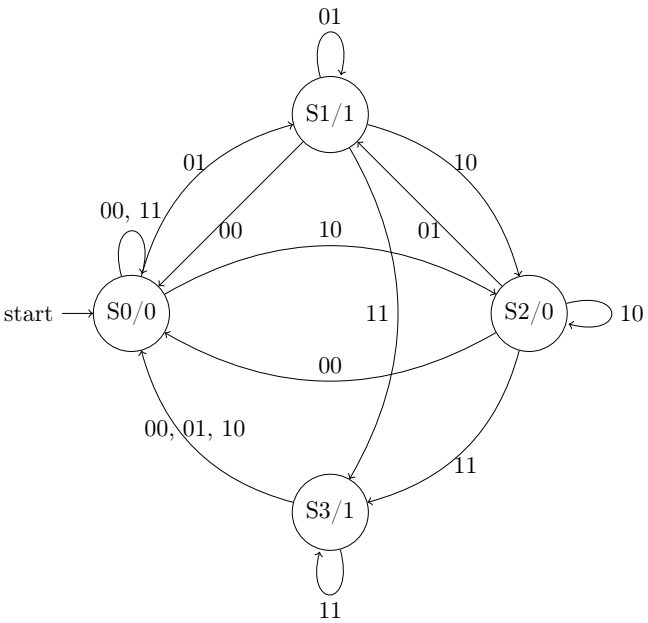
Q_0Q_1	$I=0$	$I=1$
00	0	1
01	1	1
11	1	0
10	0	0

e otteniamo: $D0 = Q_1\bar{I} + \bar{Q}_0 I$.
Dalla tabella delle transizioni della FSM risulta evidente come $U = Q_0$.
Possiamo ora procedere a disegnare il circuito logico così ottenuto:



2. Esempio 2 - Moore con FF T

Esercizio. Tradurre la seguente macchina a stati di Moore in VHDL e sintetizzarla tramite Flip Flop T:



con:

- $S = \{S0, S1, S2, S3\}$
- $I = \{00, 01, 10, 11\}$
- $U = \{0, 1\}$
- $\delta : S \times I \to S$
- $\lambda : S \to U$ (Moore)

Da Tabelle a Rete

Soluzione - VHDL. Vedi file su Webeep.

Soluzione - Sintesi. Scriviamo la tabella degli stati equivalente al diagramma della FSM:

S	$I=00$	$I=01$	$I=11$	$I=10$	U
S0	S0	S1	S0	S2	0
S1	S0	S1	S3	S2	1
S2	S0	S1	S3	S2	0
S3	S0	S0	S3	S0	1

Essendo 4 stati, possiamo codificarli (utilizzanndo la codifica naturale) attraverso 2 bit ($\lceil \log_2(4) \rceil = 2$). Convertiamo ora la tabella degli stati nella tabella delle transizioni:

Q_0Q_1	$I=00$	$I=01$	$I=11$	$I=10$	U
00	00	01	00	11	0
01	00	01	10	11	1
11	00	01	10	11	0
10	00	00	10	00	1

Recap Flip Flop T.

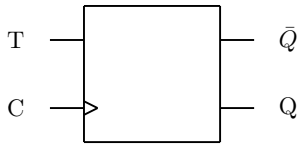


Tabella delle transizioni:

C	T	Q^*
0	-	Q
1	0	Q
1	1	$\bar{Q}$

Tabella delle eccitazioni (Q: stato corrente, Q^*: stato successivo):

Q	Q^*	T	C
0	0	0	1
0	1	1	1
1	1	0	1
1	0	1	1
0	0	-	0
1	1	-	0

Riscriviamo la tabella delle transizioni della FSM utilizzando la logica del flip flop T:

	$I=00$	$I=01$	$I=11$	$I=10$	U
Q_0Q_1	T_0T_1	T_0T_1	T_0T_1	T_0T_1	U
00	00	01	00	11	0
01	01	00	11	10	1
11	11	10	01	00	0
10	10	10	00	10	1

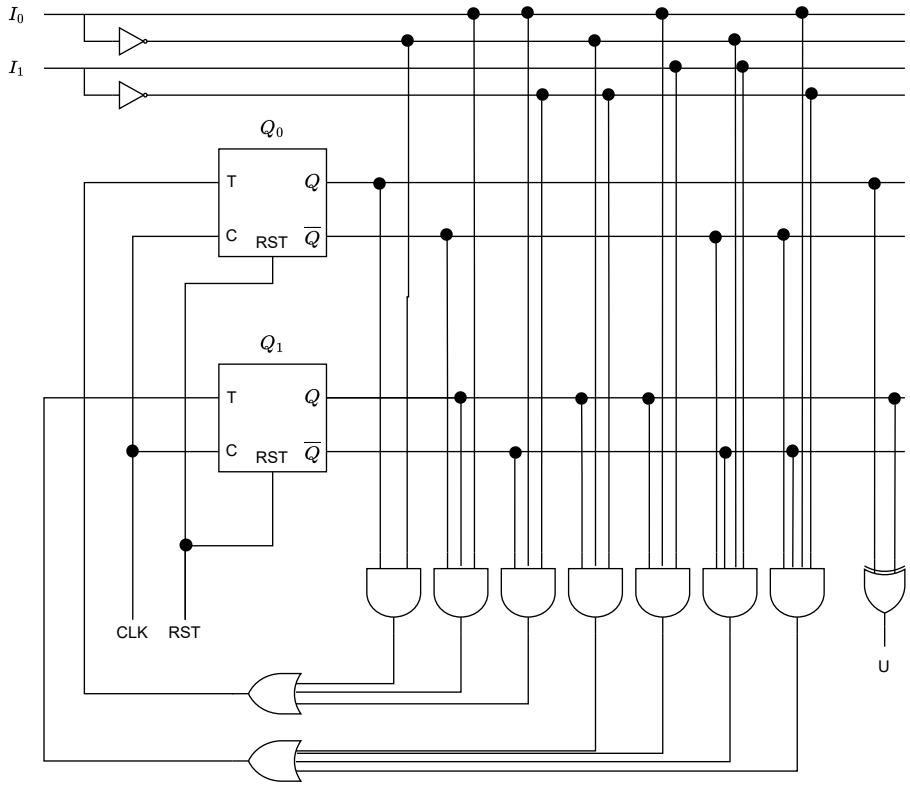
Passiamo ora alla sintesi delle funzioni logiche degli ingressi attraverso le mappe di Karnaugh:

	I_0I_1	I_0I_1	I_0I_1	I_0I_1
Q_0Q_1	00	01	11	10
00	0	0	0	1
01	0	0	1	1
11	1	1	0	0
10	1	1	0	1

e otteniamo: $T_0 = Q_0\bar{I}_0 + \bar{Q}_0Q_1I_0 + \bar{Q}_1I_0\bar{I}_1$,

	I_0I_1	I_0I_1	I_0I_1	I_0I_1
Q_0Q_1	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	1	0	1	0
10	0	0	0	0

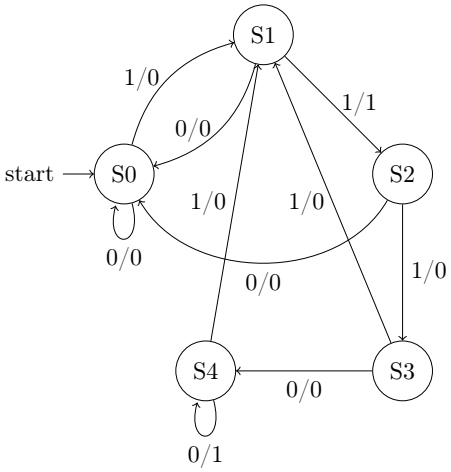
e otteniamo: $T_1 = Q_1\bar{I}_0\bar{I}_1 + Q_1I_0I_1 + \bar{Q}_0\bar{Q}_1\bar{I}_0I_1 + \bar{Q}_0\bar{Q}_1I_0\bar{I}_1$.
L'espressione logica dell'uscita è banale perchè corrisponde alla tabella di verità della porta XOR: $U = Q_0 \oplus Q_1$.
In alternativa si poteva scrivere la mappa di Karnaugh dell'uscita rispetto a Q_0 e Q_1 e trovarne gli implicanti.
Possiamo ora procedere a disegnare il circuito logico così ottenuto:



3. Esempio 3 - Mealy con FF JK

Esercizio. Tradurre la seguente macchina a stati di Mealy in VHDL e sintetizzarla tramite Flip Flop JK:

Da Tabelle a Rete



con:

- $S = \{S0, S1, S2, S3, S4\}$
- $I = \{0, 1\}$
- $U = \{0, 1\}$
- $\delta : S \times I \to S$
- $\lambda : S \times I \to U$ (Mealy)

Soluzione - VHDL. Vedi file su Webeep.

Soluzione - Sintesi. Scriviamo la tabella degli stati equivalente al diagramma della FSM:

S	$I = 0$ / U	$I = 1$ / U
S0	S0/0	S1/0
S1	S0/0	S2/1
S2	S0/0	S3/0
S3	S4/0	S1/0
S4	S4/1	S1/0

Essendo 5 stati, possiamo codificarli (utilizzanndo la codifica naturale) attraverso 3 bit ($\lceil \log_2(5) \rceil = 3$). Convertiamo ora la tabella degli stati nella tabella delle transizioni:

	$I = 0$	$I = 1$
$Q_0Q_1Q_2$	$Q_0^*Q_1^*Q_2^*/I$	$Q_0^*Q_1^*Q_2^*/I$
000	000/0	001/0
001	000/0	010/1
010	000/0	011/0
011	100/0	001/0
100	100/1	001/0

Da Tabelle a Rete

Recap Flip Flop JK.

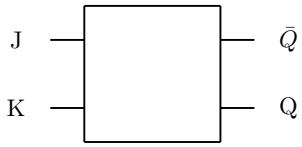


Tabella delle transizioni:

J	K	Q^*
0	0	Q
0	1	0
1	0	1
1	1	$\bar{Q}$

Tabella delle eccitazioni (Q: stato corrente, Q^*: stato successivo):

Q	Q^*	J	K
0	0	0	-
0	1	1	-
1	0	-	1
1	1	-	0

Passiamo ora alla sintesi delle funzioni logiche degli ingressi attraverso le mappe di Karnaugh.
Nota: Per ogni ingresso dei FF, dobbiamo verificare quale valore deve assumere in base alla tabella delle transizioni della FSM e delle eccitazioni del FF. Per esempio, per J_0, quando $Q_0 = 0$, $Q_1 = 0$, $Q_2 = 0$ e $I = 0$, Q_0^* deve rimanere nello stato di 0 (v. tabella transizioni FSM), quindi ci troviamo nella prima riga della tabella delle eccitazioni del FF JK, ovvero $J = 0, K = -$. Quando invece $Q_0 = 0$, $Q_1 = 1$, $Q_2 = 1$ e $I = 0$, il valore Q_0^* deve diventare 1, quindi ci troviamo nella seconda riga della tabella delle eccitazioni del FF JK, ovvero $J = 1, K = -$. Si prosegue in questo modo, ricordando che eventuali stati non esistenti producono valori DC.

J_0 :

	Q_2I	Q_2I	Q_2I	Q_2I
Q_0Q_1	00	01	11	10
00	0	0	0	0
01	0	0	0	1
11	–	–	–	–
10	–	–	–	–

$$J_0 = Q_1 Q_2 \bar{I}$$

K_0 :

	Q_2I	Q_2I	Q_2I	Q_2I
Q_0Q_1	00	01	11	10
00	–	–	–	–
01	–	–	–	–
11	–	–	–	–
10	0	1	–	–

$$K_0 = I$$

J_1 :

	Q_2I	Q_2I	Q_2I	Q_2I
Q_0Q_1	00	01	11	10
00	0	0	1	0
01	–	–	–	–
11	–	–	–	–
10	0	0	–	–

$$J_1 = Q_2 I$$

Da Tabelle a Rete

K_1 :

	Q_2I	Q_2I	Q_2I	Q_2I
Q_0Q_1	00	01	11	10
00	–	–	–	–
01	1	0	1	1
11	–	–	–	–
10	–	–	–	–

$$K_1 = Q_2 + \bar{I}$$

J_2 :

	Q_2I	Q_2I	Q_2I	Q_2I
Q_0Q_1	00	01	11	10
00	0	1	–	–
01	0	1	–	–
11	–	–	–	–
10	0	1	–	–

$$J_2 = I$$

K_2 :

	Q_2I	Q_2I	Q_2I	Q_2I
Q_0Q_1	00	01	11	10
00	–	–	1	1
01	–	–	0	1
11	–	–	–	–
10	–	–	–	–

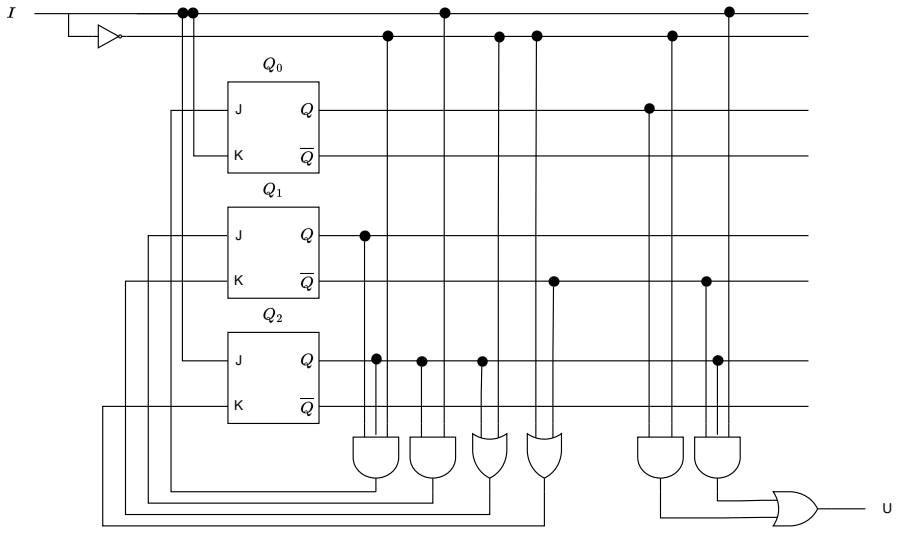
$$K_2 = \bar{Q_1} + \bar{I}$$

U :

	Q_2I	Q_2I	Q_2I	Q_2I
Q_0Q_1	00	01	11	10
00	0	0	1	0
01	0	0	0	0
11	–	–	–	–
10	1	0	–	–

$$U = Q_0\bar{I} + \bar{Q_1}Q_2I$$

Possiamo ora procedere a disegnare il circuito logico così ottenuto:



Metodo Score – Fattorizzazione

ESERCIZIO 3

Date la seguente funzione di quattro variabli ed un solo bit di uscita $F(A, B, C, D)$:

$$F(A, B, C, D) = A \cdot B \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot \overline{D} + \overline{A} \cdot C \cdot D + \overline{B} \cdot C \cdot \overline{D}$$

Si richiede, commentando opportunamente la risposta, di fattorizzare l'espressione facendo uso delle tecniche viste a lezione, commentando i vari passaggi e le scelte intraprese. Si riportino tutte le tabelle costruite e le espressioni logiche intermedie e finali.

SOLUZIONE
Un facile controllo di correttezza del risultato si può fare svilluppando i prodotti dell'espressione fattorizzata per vedere se si riottengono i medesimi dell'espressione di partenza

